Execution In Shell:

```
sravan@PRASADSTOREA:/mnt/c/Users/prasa/OneDrive/Desktop/OS_SDDDD$ cd OS_Projects_G5/
sravan@PRASADSTOREA:/mnt/c/Users/prasa/OneDrive/Desktop/OS_SDDDD/OS_Projects_G5$ cd Project_2/
sravan@PRASADSTOREA:/mnt/c/Users/prasa/OneDrive/Desktop/OS_SDDDD/OS_Projects_G5/Project_2$ cd custom_commands/
sravan@PRASADSTOREA:/mnt/c/Users/prasa/OneDrive/Desktop/OS_SDDDD/OS_Projects_G5/Project_2/custom_commands$ ls
custom_cat  custom_cat.c  custom_cp.c  custom_grep.c  custom_ls.c  custom_mv.c  custom_wc.c  sample.txt  sample2.txt  sample_cat.txt
sravan@PRASADSTOREA:/mnt/c/Users/prasa/OneDrive/Desktop/OS_SDDDD/OS_Projects_G5/Project_2/custom_commands$ ls
custom_cat  custom_cat.c  custom_cp.c  custom_grep.c  custom_ls.c  custom_mv.c  custom_wc.c  sample2.txt  sample_cat.txt  sample_grep.txt
sravan@PRASADSTOREA:/mnt/c/Users/prasa/OneDrive/Desktop/OS_SDDDD/OS_Projects_G5/Project_2/custom_commands$ gcc custom_grep.c -o custom_grep
sravan@PRASADSTOREA:/mnt/c/Users/prasa/OneDrive/Desktop/OS_SDDDD/OS_Projects_G5/Project_2/custom_commands$ cd ..
sravan@PRASADSTOREA:/mnt/c/Users/prasa/OneDrive/Desktop/OS_SDDDD/OS_Projects_G5/Project_2$ cd Shell/
sravan@PRASADSTOREA:/mnt/c/Users/prasa/OneDrive/Desktop/OS_SDDDD/OS_Projects_G5/Project_2/Shell$ ls
shell.c
sravan@PRASADSTOREA:/mnt/c/Users/prasa/OneDrive/Desktop/OS_SDDDD/OS_Projects_G5/Project_2/Shell$ gcc shell.c -o shell
sravan@PRASADSTOREA:/mnt/c/Users/prasa/OneDrive/Desktop/OS_SDDDD/OS_Projects_G5/Project_2/Shell$ ls
shell  shell.c
sravan@PRASADSTOREA:/mnt/c/Users/prasa/OneDrive/Desktop/OS_SDDDD/OS_Projects_G5/Project_2/Shell$ ./shell
G5_Shell> ../custom_commands/custom_grep -ni sleep ../custom_commands/sample_grep.txt
3:And miles to go before I sleep,
4:And Miles to go before I Sleep.
16:Are you asleep or just sleeping?
G5_Shell> ../custom_commands/custom_grep -n sleep ../custom_commands/sample_grep.txt
3:And miles to go before I sleep,
16:Are you asleep or just sleeping?
G5_Shell> ../custom_commands/custom_grep -ic sleep ../custom_commands/sample_grep.txt
3
G5_Shell>
```

```c
#include<stdio.h>
#include<string.h>
#include<ctype.h>
#define MAX_LINE 1024

// Convert string to lowercase in-place
void to_lower(char *s) {
    for(; *s; s++)
        *s = tolower((unsigned char)*s);
}

// Returns 1 if pattern found in text, 0 otherwise
int contains(const char *text,const char *pattern) {
    int tlen=strlen(text);
    int plen=strlen(pattern);
    int i,j;

    for(i=0; i<=tlen-plen; i++) {
        for(j=0; j<plen; j++) {
            if(text[i+j]!=pattern[j])
                break;
        }
        if(j==plen)
            return 1;
    }
    return 0;
}
```

```c
int main(int argc,char *argv[]) {
    int flag_i=0;   // -i: ignore case
    int flag_n=0;   // -n: show line numbers
    int flag_c=0;   // -c: count only

    int argi=1;

    // Parse flags
    while(argi<argc && argv[argi][0]=='-') {
        char *opt=argv[argi]+1;
        while(*opt) {
            if(*opt=='i') {
                flag_i=1;
            } else if(*opt=='n') {
                flag_n=1;
            } else if(*opt=='c') {
                flag_c=1;
            } else {
                fprintf(stderr,"custom_grep: unknown option '-%c'\n", *opt);
                return 1;
            }
            opt++;
        }
        argi++;
    }

    // Need at least a pattern
    if (argi>=argc) {
        fprintf(stderr,"Usage: custom_grep [-n] [-i] [-c] \"PATTERN\" [FILE]\n");
        fprintf(stderr,"  Tip: Always quote PATTERN to avoid shell misinterpretation.\n");
        return 1;
    }
```

```c
    char pattern[MAX_LINE];
    strncpy(pattern,argv[argi++],MAX_LINE-1);
    pattern[MAX_LINE-1]='\0';

    // Open file or use stdin
    FILE *fp=stdin;
    if(argi<argc) {
        fp=fopen(argv[argi],"r");
        if(!fp) {
            fprintf(stderr,"Error: custom_grep cannot open '%s'\n",argv[argi]);
            return 1;
        }
        argi++; // advance past the filename

        // Warn about extra arguments that will be ignored
        if (argi<argc) {
            fprintf(stderr,"custom_grep: warning: extra arguments ignored:");
            while (argi<argc)
                fprintf(stderr, " '%s'",argv[argi++]);
            fprintf(stderr,"\n");
        }
    } else {
        // No file given — reading from standard input
        fprintf(stderr, "custom_grep: no file specified, reading from stdin (Ctrl+D to end)\n");
    }

    // Lower the pattern once if -i
    char pat_cmp[MAX_LINE];
    strncpy(pat_cmp,pattern,MAX_LINE-1);
    pat_cmp[MAX_LINE-1]='\0';
    if(flag_i)
        to_lower(pat_cmp);
```

```c
    char line[MAX_LINE];
    int lineno=0;
    int count=0;

    while(fgets(line,MAX_LINE,fp)) {
        lineno++;
        // Strip trailing newline for cleaner output
        int len=strlen(line);
        if(len>0 && line[len-1]=='\n')
            line[len-1]='\0';

        // Make a copy to compare (lowered if -i)
        char line_cmp[MAX_LINE];
        strncpy(line_cmp,line,MAX_LINE-1);
        line_cmp[MAX_LINE-1]='\0';
        if(flag_i)
            to_lower(line_cmp);

        if(!contains(line_cmp,pat_cmp))
            continue;

        count++;

        if(!flag_c) { // if count is required no need print the lines
            if(flag_n)
                printf("%d:",lineno);
            printf("%s\n",line);
        }
    }
```

```c
    if(flag_c)
        printf("%d\n",count);

    if(count==0)
        printf("No match found.\n");

    if(fp!=stdin)
        fclose(fp);

    return ((count>0)?0:1);
}
```